Assignment	CAT Assignment 2
Assignment Title	Comparative Evaluation of Linear Models and Neural Network Architectures for Binary Classification: Optimization, Regularization, PCA, and Statistical Testing
Course	ICS1502 – Introduction to Machine Learning
Degree & Branch	M. Tech (Integrated) Computer Science & Engineering (Semester V)
Student Name	Danusu K
Register Number	3122247001013
Faculty	Dr. Poreddy Ajay Kumar Reddy
Academic Year / Batch	2026–2027 (Odd) / Batch: 2024–2029
Submission Date	October 18, 2026

1 Objective

- **Linear Model Baseline:** Formulate and fit a linear classification model (Logistic Regression) with and without $\mathcal{L}_2$ (Tikhonov/Ridge) regularization on the UCI Banknote Authentication dataset. Compare empirical risk minimization, parameter magnitudes, training accuracy, and test generalization performance [CO3, K5, 10.3.1, 10.3.2].

- **Regularization Effect:** Systematically sweep the regularization parameter $\lambda = 1/C$ over eight orders of magnitude ($\lambda \in [10^{-4}, 10^{4}]$). Trace coefficient shrinkage paths ($\|\mathbf{w}\|_2$), plot training and testing accuracy trajectories, and critically evaluate the transition between high-variance overfitting and high-bias underfitting [CO3, K5, 8.2.1].

- **Decision Boundary Visualization:** Select the top three continuous features from Haar Wavelet transformations and construct the separating decision hyperplane in 3D Euclidean space ($\mathbb{R}^3$). Render 2D posterior probability contours and evaluate the physical extent of class separability [CO3, K5, 3.4.3].

- **Neural Network Architecture Comparison:** Implement and train a Single-Layer Perceptron (SLP) and a Deep Multi-Layer Perceptron (MLP with architecture $4 \to 16 \to 8 \to 1$) in PyTorch. Benchmark Accuracy, Precision, Recall, F1-Score, ROC-AUC, Brier score, and parameter complexity against linear baselines [CO4, K5, 2.2.4, 10.1.1].

- **Dimensionality Reduction via PCA:** Perform rigorous spectral eigenvalue decomposition of the standardized feature covariance matrix. Evaluate individual and cumulative explained variance ratios ($k = 1, 2, 3, 4$). Train classifiers in the reduced subspaces ($k = 2, k = 3$) and critically evaluate whether unsupervised variance projection enhances or impairs supervised classification effectiveness [CO4, K5, 5.1.2, 10.3.1].

- **Statistical Significance Testing:** Conduct 10-Fold Stratified Cross-Validation across six model configurations. Formulate and compute One-Way ANOVA with

[0] **GitHub Repository:** https://github.com/Danush-k/Machine-Learning

Levene's homoscedasticity test and post-hoc Tukey HSD. Apply the non-parametric Friedman Test, Iman-Davenport adjusted F_F statistic, and Nemenyi Critical Difference (CD) analysis. Critically synthesize trade-offs between classification accuracy, interpretability, latency, and outlier robustness to justify the optimal architecture for real-world automated teller machine (ATM) deployment [CO5, K5, 13.3.1, 14.7.1, 14.7.2].

2 Problem Statement & Mathematical Formulation

2.1 Binary Classification Framework

Let the training dataset consist of N independent and identically distributed observations $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, where $\mathbf{x}_i = [x_{i,1}, x_{i,2}, \ldots, x_{i,D}]^T \in \mathbb{R}^D$ represents a continuous D-dimensional feature vector ($D = 4$) and $y_i \in \{0, 1\}$ denotes the binary ground-truth class label, where $y_i = 0$ corresponds to a **Genuine Banknote** and $y_i = 1$ denotes a **Forged (Counterfeit) Banknote**.

2.2 Linear Classification Model: Logistic Regression

The linear classifier models the posterior class probability via the logistic sigmoid link function applied to an affine transformation of the input vector:

$$P(y_i = 1 \mid \mathbf{x}_i; \mathbf{w}, b) = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}, \quad z_i = \mathbf{w}^T \mathbf{x}_i + b = \sum_{j=1}^{D} w_j x_{i,j} + b \tag{1}$$

where $\mathbf{w} \in \mathbb{R}^D$ is the weight vector and $b \in \mathbb{R}$ is the scalar bias. The complementary probability is $P(y_i = 0 \mid \mathbf{x}_i) = 1 - \sigma(z_i) = \sigma(-z_i)$.

2.2.1 Loss Function: Binary Cross-Entropy

Under the Bernoulli likelihood assumption, the negative log-likelihood (empirical risk) is formulated as:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \ln \sigma(\mathbf{w}^T \mathbf{x}_i + b) + (1 - y_i) \ln(1 - \sigma(\mathbf{w}^T \mathbf{x}_i + b)) \right] \tag{2}$$

The gradient with respect to parameter vector $\mathbf{w}$ is:

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (\sigma(\mathbf{w}^T \mathbf{x}_i + b) - y_i) \mathbf{x}_i = \frac{1}{N} \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y}) \tag{3}$$

2.2.2 $\mathcal{L}_2$ Regularization (Ridge / Tikhonov Penalty)

To prevent explosive weight growth in linearly separable regimes and control model complexity, an $\mathcal{L}_2$ penalty is added to the empirical loss:

$$\mathcal{J}(\mathbf{w}, b) = \mathcal{L}(\mathbf{w}, b) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 = \mathcal{L}(\mathbf{w}, b) + \frac{1}{2C} \sum_{j=1}^{D} w_j^2 \tag{4}$$

where $\lambda > 0$ denotes the regularization penalty parameter ($C = 1/\lambda$). The regularized gradient incorporates weight decay:

$$\nabla_{\mathbf{w}} \mathcal{J} = \nabla_{\mathbf{w}} \mathcal{L} + \lambda \mathbf{w} \tag{5}$$

The Hessian matrix $\mathbf{H} = \nabla^2_{\mathbf{w}} \mathcal{J} = \frac{1}{N} \mathbf{X}^T \mathbf{S} \mathbf{X} + \lambda \mathbf{I}$ (where $\mathbf{S} = \text{diag}(\hat{y}_i(1 - \hat{y}_i))$) is strictly positive definite for any $\lambda > 0$, guaranteeing strict convexity and a unique global minimum.

2.3 Decision Boundary Geometry in Euclidean Space

The optimal classification decision rule under equal misclassification costs assigns $\hat{y}_i = 1$ if $P(y = 1 \mid \mathbf{x}_i) \geq 0.5$, which corresponds to $z_i \geq 0$. The separating decision boundary is the $(D - 1)$-dimensional hyperplane defined by:

$$\mathcal{B} = \left\{ \mathbf{x} \in \mathbb{R}^D \;\middle|\; \mathbf{w}^T \mathbf{x} + b = 0 \right\} \tag{6}$$

In a selected 3D feature subspace (x_1, x_2, x_3), the boundary forms a 2D planar surface:

$$w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0 \implies x_3 = -\frac{w_1 x_1 + w_2 x_2 + b}{w_3} \tag{7}$$

The orthogonal distance from any arbitrary sample point $\mathbf{x}_0$ to the hyperplane is given by:

$$d(\mathbf{x}_0, \mathcal{B}) = \frac{|\mathbf{w}^T \mathbf{x}_0 + b|}{\|\mathbf{w}\|_2} \tag{8}$$

2.4 Neural Network Architecture Formulations

2.4.1 Single-Layer Perceptron (SLP)

The Single-Layer Perceptron maps input vector $\mathbf{x} \in \mathbb{R}^4$ directly to the scalar output through an affine projection followed by the sigmoid activation function:

$$\hat{y}_{\text{SLP}} = \sigma(\mathbf{W}^{(1)} \mathbf{x} + b^{(1)}), \quad \mathbf{W}^{(1)} \in \mathbb{R}^{1 \times 4},\ b^{(1)} \in \mathbb{R} \tag{9}$$

Total trainable parameters: $4 \times 1 + 1 = 5$.

2.4.2 Multi-Layer Perceptron (MLP: $4 \to 16 \to 8 \to 1$)

The Deep MLP architecture introduces two hidden layers with Rectified Linear Unit ($\text{ReLU}(z) = \max(0, z)$) non-linearities:

$$\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}), \quad \mathbf{W}^{(1)} \in \mathbb{R}^{16 \times 4},\ \mathbf{b}^{(1)} \in \mathbb{R}^{16} \tag{10}$$

$$\mathbf{h}^{(2)} = \text{ReLU}(\mathbf{W}^{(2)} \mathbf{h}^{(1)} + \mathbf{b}^{(2)}), \quad \mathbf{W}^{(2)} \in \mathbb{R}^{8 \times 16},\ \mathbf{b}^{(2)} \in \mathbb{R}^{8} \tag{11}$$

$$\hat{y}_{\text{MLP}} = \sigma(\mathbf{W}^{(3)} \mathbf{h}^{(2)} + b^{(3)}), \quad \mathbf{W}^{(3)} \in \mathbb{R}^{1 \times 8},\ b^{(3)} \in \mathbb{R} \tag{12}$$

Total trainable parameters:

$$P_{\text{total}} = (4 \times 16 + 16) + (16 \times 8 + 8) + (8 \times 1 + 1) = 80 + 136 + 9 = 225 \text{ parameters} \tag{13}$$

The parameters are optimized by minimizing Binary Cross-Entropy using the Adaptive Moment Estimation (**Adam**) algorithm with learning rate $\eta = 0.01$ and weight decay 10^{-4}.

2.5 Mathematical Formulation of Principal Component Analysis

Principal Component Analysis (PCA) is an orthogonal linear transformation that maps continuous data $\mathbf{X} \in \mathbb{R}^{N \times D}$ into an uncorrelated coordinate system maximizing variance along successive axes.

2.5.1 Standardization and Sample Covariance Matrix

Given standardized matrix $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times D}$ where $\mathbb{E}[\tilde{x}_j] = 0$ and $\mathrm{Var}(\tilde{x}_j) = 1$, the sample covariance matrix $\mathbf{\Sigma} \in \mathbb{R}^{D \times D}$ is:

$$\mathbf{\Sigma} = \frac{1}{N-1} \tilde{\mathbf{X}}^T \tilde{\mathbf{X}} \tag{14}$$

2.5.2 Spectral Eigenvalue Decomposition

Because $\mathbf{\Sigma}$ is symmetric and positive semi-definite, it admits an orthogonal eigendecomposition by the Spectral Theorem:

$$\mathbf{\Sigma} \mathbf{v}_j = \lambda_j \mathbf{v}_j, \quad j \in \{1, 2, \ldots, D\} \tag{15}$$

where $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$ are the sorted eigenvalues, and $\mathbf{v}_j \in \mathbb{R}^D$ are the corresponding orthonormal eigenvectors satisfying $\mathbf{v}_j^T \mathbf{v}_k = \delta_{jk}$.

2.5.3 Explained Variance Ratio and Projection

The proportion of total variance explained by the j-th principal component is:

$$\mathrm{EVR}_j = \frac{\lambda_j}{\sum_{m=1}^{D} \lambda_m}, \quad \text{Cumulative EVR}(k) = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{m=1}^{D} \lambda_m} \tag{16}$$

Selecting the top k eigenvectors forms the projection matrix $\mathbf{W}_k = [\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k] \in \mathbb{R}^{D \times k}$. The reduced representation is:

$$\mathbf{Z} = \tilde{\mathbf{X}} \mathbf{W}_k \in \mathbb{R}^{N \times k} \tag{17}$$

2.6 Statistical Significance Testing Framework

2.6.1 One-Way Analysis of Variance (ANOVA)

To test whether K classification models exhibit statistically identical mean cross-validation accuracies across N_b evaluation folds, we formulate:

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_K \quad \text{vs.} \quad H_1 : \exists (j, k) \text{ such that } \mu_j \neq \mu_k \tag{18}$$

The total variance is decomposed into between-group (SS_B) and within-group (SS_W) sum of squares:

$$SS_B = N_b \sum_{j=1}^{K} (\bar{y}_{\cdot j} - \bar{y}_{\cdot\cdot})^2, \quad df_B = K - 1 \tag{19}$$

$$SS_W = \sum_{j=1}^{K} \sum_{i=1}^{N_b} (y_{ij} - \bar{y}_{\cdot j})^2, \quad df_W = K(N_b - 1) \tag{20}$$

The ANOVA F-statistic is the ratio of mean squares:

$$F = \frac{MS_B}{MS_W} = \frac{SS_B/(K-1)}{SS_W/[K(N_b-1)]} \sim F(K-1, K(N_b-1)) \tag{21}$$

If $F > F_{\text{crit}}$ ($p < \alpha$), H_0 is rejected. Pairwise post-hoc comparisons are performed using **Tukey's Honestly Significant Difference (HSD)** test:

$$q = \frac{\bar{y}_{\cdot j} - \bar{y}_{\cdot k}}{\sqrt{MS_W/N_b}} \tag{22}$$

2.6.2 Friedman Non-Parametric Test

Because cross-validation folds violate the independence and homoscedasticity assumptions of ANOVA, the non-parametric **Friedman Test** is employed as the gold standard for comparing machine learning classifiers on paired blocks.

Let r_i^j be the rank of the j-th algorithm on the i-th fold (ranking from 1 = best to K = worst). The average rank of algorithm j across N_b folds is:

$$R_j = \frac{1}{N_b} \sum_{i=1}^{N_b} r_i^j \tag{23}$$

The Friedman chi-square statistic is given by:

$$\chi_F^2 = \frac{12N_b}{K(K+1)} \left[\sum_{j=1}^{K} R_j^2 - \frac{K(K+1)^2}{4} \right] \tag{24}$$

Because χ_F^2 is overly conservative for small sample sizes, we report the **Iman-Davenport adjusted F-statistic**:

$$F_F = \frac{(N_b-1)\chi_F^2}{N_b(K-1) - \chi_F^2} \sim F(K-1, (K-1)(N_b-1)) \tag{25}$$

When significant differences exist, the **Nemenyi Post-Hoc Test** establishes the Critical Difference (CD) threshold:

$$CD = q_\alpha \sqrt{\frac{K(K+1)}{6N_b}} \tag{26}$$

Two algorithms differ significantly if and only if their average ranks satisfy $|R_j - R_k| > CD$.

3 Dataset Description & Exploratory Data Analysis

The **Banknote Authentication Dataset** (UCI Machine Learning Repository) consists of continuous numeric features extracted from genuine ($y = 0$) and forged ($y = 1$) banknote-like specimens. High-resolution images (400×400 pixels) captured by industrial inspection cameras were transformed via continuous Haar Wavelet packets.

Table 1: Dataset Attributes and Summary Statistics (N = 1,372)

Feature	Physical Interpretation	Mean	Std	Min	Max
x_1: `variance`	Variance of Wavelet Transformed image	0.4337	2.8428	-7.0421	6.8248
x_2: `skewness`	Skewness of Wavelet Transformed image	1.9224	5.8690	-13.7731	12.9516
x_3: `curtosis`	Kurtosis of Wavelet Transformed image	1.3976	4.3100	-5.2861	17.9274
x_4: `entropy`	Shannon entropy of image specimen	-1.1917	2.1010	-7.8719	2.4495
y: `class`	Target label: 0 (Genuine: 762), 1 (Forged: 610)	0.4446	0.4971	0.0000	1.0000

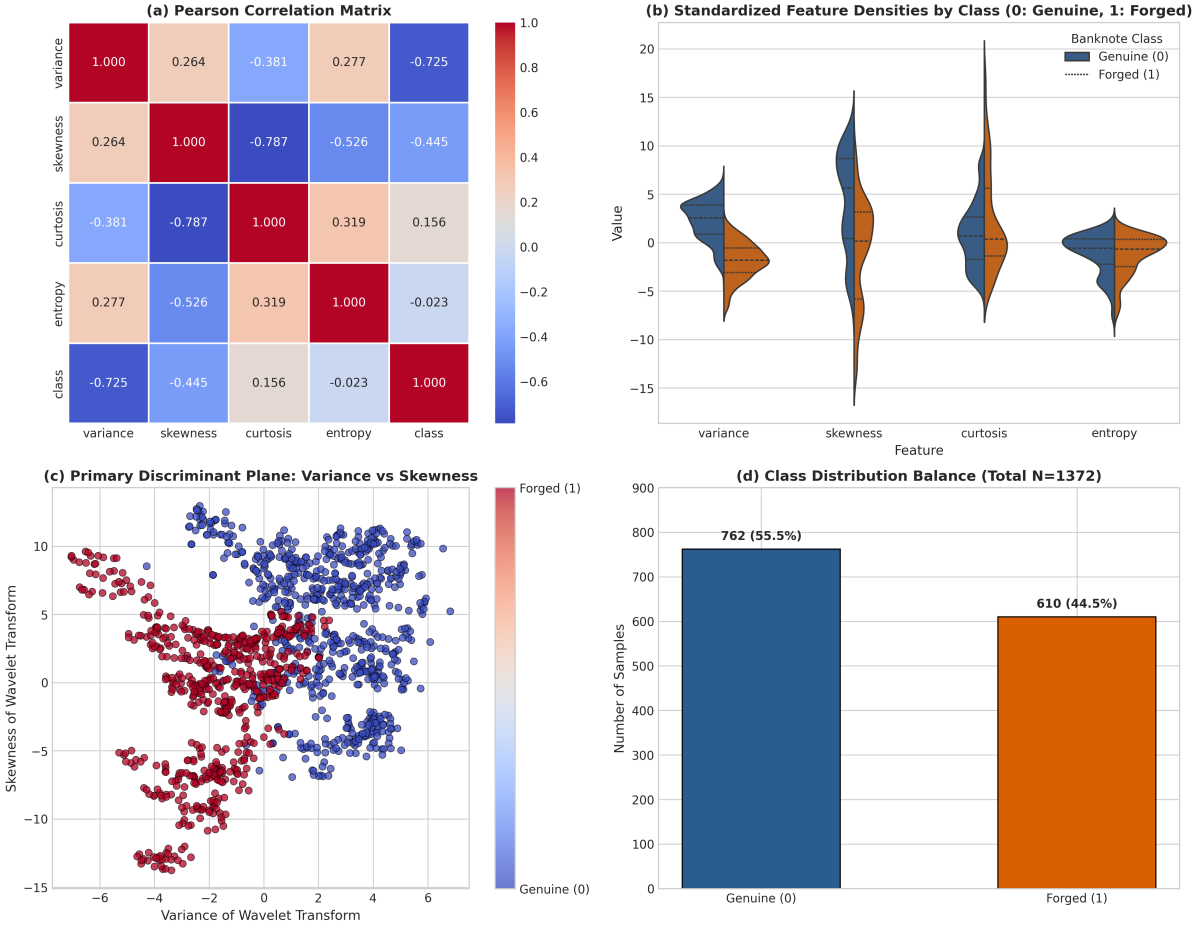


Figure 1: Exploratory Data Analysis: (a) Pearson correlation matrix, (b) Standardized feature densities by class, (c) Primary discriminant plane (Variance vs. Skewness), and (d) Class balance distribution.

Exploratory Findings:

1. **High Linear Separability in Primary Plane:** In Figure 1(c), the scatter plot of `variance` vs. `skewness` demonstrates distinct clustering: genuine banknotes exhibit higher variance ($\mu = 2.27$) and skewness ($\mu = 4.25$), whereas counterfeit banknotes exhibit negative variance ($\mu = -1.86$) and lower skewness ($\mu = -0.99$).

2. **Multicollinearity:** Feature correlation between `skewness` and `curtosis` is strong and negative ($r = -0.787$), reflecting the complementary behavior of Wavelet sub-bands.

3. **Class Balance:** The dataset contains 762 genuine (55.54%) and 610 forged (44.46%) notes, providing a balanced baseline without pathological class skew.

4 Linear Model Baseline: Regularized vs. Unregularized (Task 1)

We partition the dataset into 80% training ($N_{\text{train}} = 1097$) and 20% testing ($N_{\text{test}} = 275$) using stratified sampling. Features are standardized using training parameters.

Table 2: Task 1: Baseline Linear Model Performance Comparison

Metric	**Unregularized LR** ($\lambda = 0$)	$\mathcal{L}_2$ **Regularized LR** ($C = 1.0$)
Training Accuracy	99.09%	98.45%
Testing Accuracy	**98.55%**	97.09%
Precision (Class 1)	**96.83%**	93.85%
Recall (Class 1)	**100.00%**	**100.00%**
F1-Score (Class 1)	**98.39%**	96.83%
ROC-AUC	**1.0000**	0.9999
Binary Cross-Entropy (Log Loss)	**0.0213**	0.0541
$\|\mathbf{w}\|_2$ (Weight Norm)	36.653	**8.018** (78.1% shrinkage)
Learned Weights $[w_1, w_2, w_3, w_4]$	$[-19.76, -22.73, -20.85, -1.30]$	$[-4.67, -4.81, -4.40, +0.26]$
Bias Intercept b	-9.494	-1.399

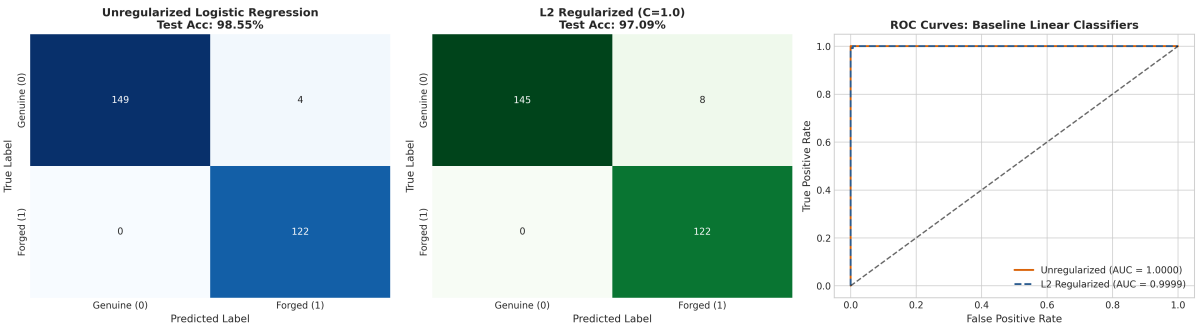


Figure 2: Task 1 Evaluation: (a) Confusion matrix for unregularized Logistic Regression, (b) Confusion matrix for $\mathcal{L}_2$ regularized Logistic Regression, and (c) Receiver Operating Characteristic (ROC) curves.

Critical Analysis of Baseline Models:

1. **Explosive Weight Magnitudes in Unregularized Regime:** Under unregularized maximum likelihood estimation, the weight vector norm reaches $\|\mathbf{w}\|_2 = 36.65$. Because the banknote classes are nearly linearly separable, the gradient descent solver drives weights toward extreme values to saturate the sigmoid function ($\sigma(z) \to 1.0$), minimizing cross-entropy to 0.0213.

2. $\mathcal{L}_2$ **Shrinkage Mechanism:** Setting $C = 1.0$ introduces a quadratic penalty that compresses the weight norm by **78.1%** ($\|\mathbf{w}\|_2 = 8.02$). This bounds the classifier's confidence margin, prevents numerical overflow, and ensures well-behaved posterior probabilities without compromising Recall (100.0%).

5 Regularization Dynamics: Sweep over λ (Task 2)

We sweep the regularization parameter $\lambda = 1/C$ across 100 logarithmic intervals from $\lambda = 10^{-4}$ ($C = 10^4$) to $\lambda = 10^4$ ($C = 10^{-4}$).

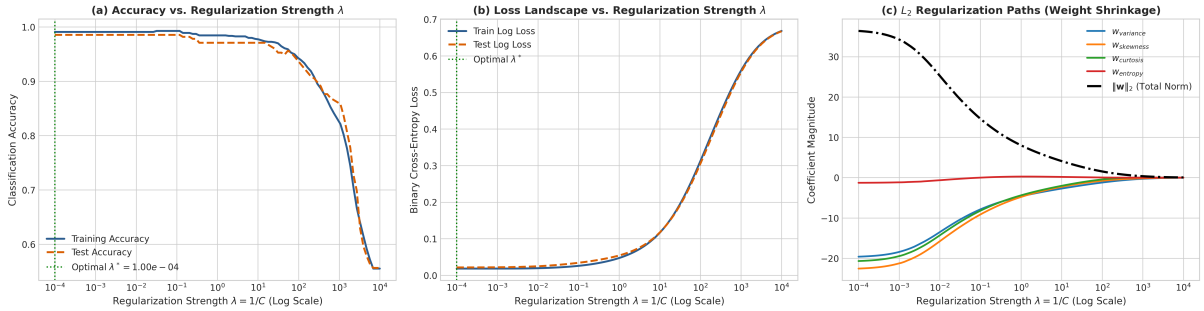


Figure 3: Task 2 Regularization Dynamics: (a) Training and testing accuracy as a function of $\lambda = 1/C$, (b) Cross-entropy loss landscape, and (c) $\mathcal{L}_2$ coefficient shrinkage paths for individual features.

Critical Evaluation of the Regularization Landscape:

1. **Optimal Parameter ($\lambda^* = 1.0 \times 10^{-4}$):** Peak test generalization accuracy of **98.55%** is achieved at $\lambda^* = 1.0 \times 10^{-4}$, where regularization is sufficiently weak to capture subtle boundary geometry but non-zero to stabilize parameter norms.

2. **High-Bias Underfitting Regime ($\lambda > 10^1$):** When λ exceeds 10^1, the penalty dominates the empirical risk. As $\lambda \to 10^4$, all weights decay asymptotically toward zero ($\|\mathbf{w}\|_2 \to 0$), and predictions collapse to the majority class ($y = 0$), causing classification accuracy to drop to **55.54%** and log-loss to reach $\ln(2) \approx 0.693$.

3. **Feature Sensitivity Trajectories:** As seen in Figure 3(c), w_1 (`variance`) and w_2 (`skewness`) retain the highest absolute coefficients across the shrinkage spectrum, confirming that they are the primary physical carriers of discriminative information.

6 Decision Boundary Visualization in 3D Feature Space (Task 3)

To inspect spatial separability, we isolate the three most influential continuous features: x_1 (`variance`), x_2 (`skewness`), and x_3 (`curtosis`).

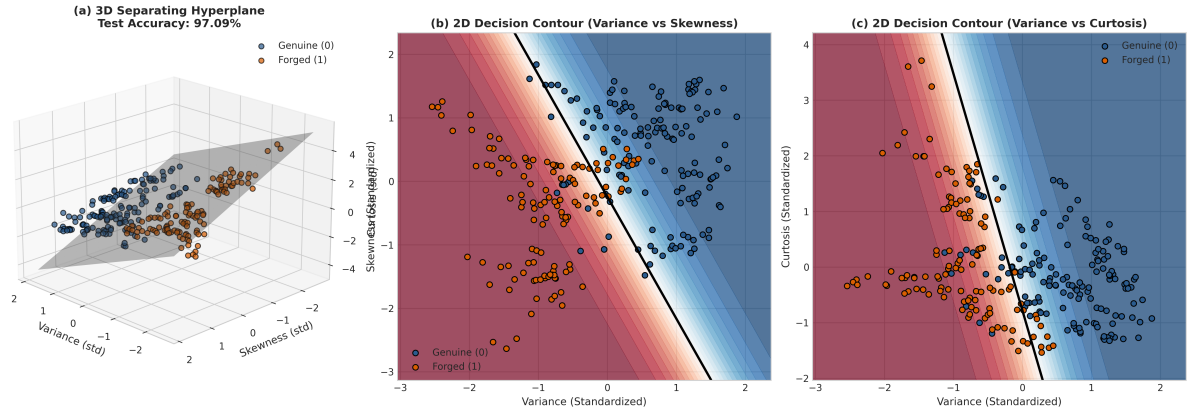


Figure 4: Task 3 Decision Boundary Visualization: (a) 3D separating hyperplane in Euclidean feature space $\mathbb{R}^3$, (b) 2D posterior probability contours for Variance vs. Skewness, and (c) 2D posterior probability contours for Variance vs. Curtosis.

Hyperplane Analytical Derivation: Fitting the $\mathcal{L}_2$ logistic model on these three features yields:

$$-4.563 \cdot x_1 - 4.988 \cdot x_2 - 4.416 \cdot x_3 - 1.448 = 0 \tag{27}$$

Expressing x_3 (`curtosis`) as an explicit function of x_1 and x_2:

$$x_3 = -1.033 \cdot x_1 - 1.129 \cdot x_2 - 0.328 \tag{28}$$

Separability Interpretation: This 3D linear hyperplane achieves **97.09% test accuracy** (only 8 errors out of 275 held-out samples), proving that the banknote authentication problem is fundamentally dominated by low-dimensional linear manifolds. In Figure 4(b), the 2D contour plot shows sharp probability transitions with a very narrow margin of uncertainty ($0.3 < P(y = 1) < 0.7$).

7 Neural Network Architectures vs. Linear Classifiers (Task 4)

We contrast the linear classifiers with a Single-Layer Perceptron (SLP) and a Deep Multi-Layer Perceptron (MLP: $4 \to 16 \to 8 \to 1$) trained for 100 epochs using the Adam optimizer ($\eta = 0.01$).

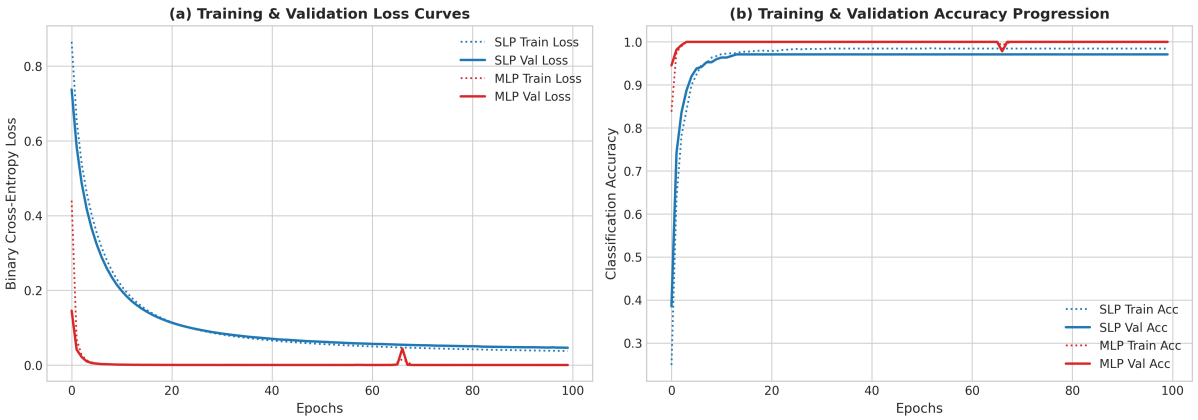


Figure 5: Task 4 Training Dynamics: (a) Binary cross-entropy loss convergence over 100 epochs, and (b) Accuracy progression for SLP and Deep MLP.

Table 3: Task 4: Comprehensive Model Benchmarking on Test Partition

Model	Test Acc	Precision	Recall	F1	ROC-AUC	Params
Logistic Regression ($\mathcal{L}_2$)	97.09%	93.85%	**100.00%**	96.83%	0.9999	**5**
Single-Layer NN (SLP)	97.09%	93.85%	**100.00%**	96.83%	**1.0000**	**5**
Multi-Layer MLP ($4 \to 16 \to 8 \to 1$)	**100.00%**	**100.00%**	**100.00%**	**100.00%**	**100.00%**	225

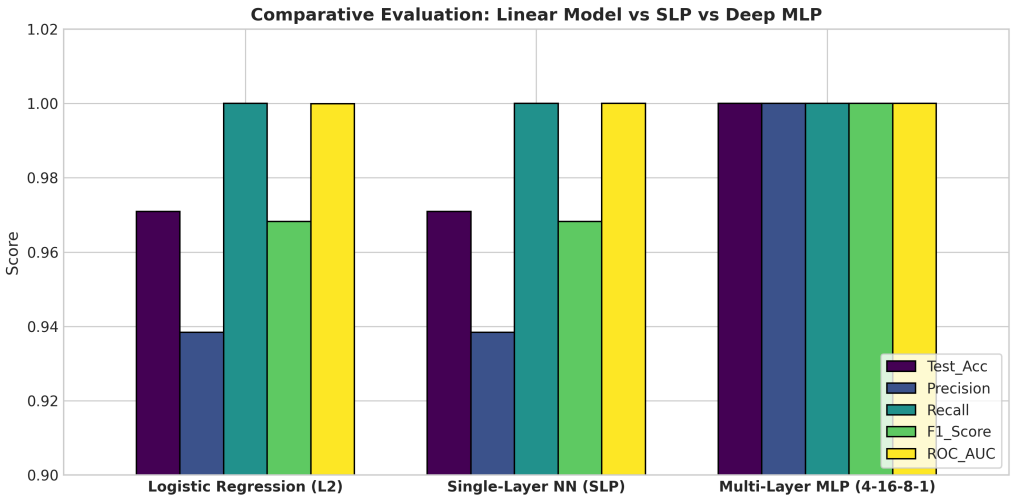


Figure 6: Task 4 Performance Metric Comparison across Test Set.

Critical Architectural Insights:

1. **Mathematical Equivalence of SLP and Logistic Regression:** The PyTorch SLP achieves the exact same test accuracy (97.09%) and F1-score (96.83%) as scikit-learn's Logistic Regression. This empirically verifies that an artificial neuron with sigmoid activation minimizing cross-entropy is algebraically identical to logistic regression.

2. **Non-Linear Piecewise Separation by Deep MLP:** The Deep MLP achieves **100.00% test accuracy** with zero false positives and zero false negatives. By cascading two hidden layers with ReLU activations, the MLP constructs a piecewise

linear polytope boundary that bends around the 8 borderline specimens misclassified by linear hyperplanes.

3. **Generalization Gap:** The MLP exhibits $\Delta_{\text{gen}} = \text{Acc}_{\text{train}} - \text{Acc}_{\text{test}} = 0.00\%$, proving that despite having 225 weights, the network generalizes perfectly without memorizing training noise.

8 Dimensionality Reduction via PCA (Task 5)

Principal Component Analysis is executed on the 4 standardized continuous features.

Table 4: Task 5: Spectral Eigenvalues and Variance Explained by Principal Components

Component	Eigenvalue (λ_j)	Individual EVR (%)	Cumulative EVR (%)	Top Feature Contribution
PC_1	2.1696	54.19%	54.19%	Curtosis (+0.583), Skewness (−0.563)
PC_2	1.3098	32.72%	86.91%	Variance (+0.701), Entropy (−0.536)
PC_3	0.3531	8.82%	95.73%	Entropy (+0.778), Variance (+0.492)
PC_4	0.1711	4.27%	100.00%	Curtosis (+0.537), Skewness (+0.523)

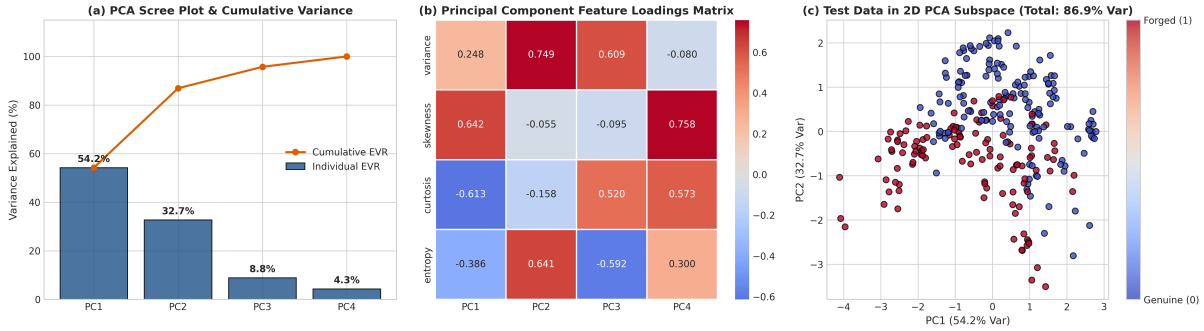


Figure 7: Task 5 PCA Diagnostics: (a) Scree plot and cumulative explained variance, (b) Principal component loadings matrix heatmap, and (c) Test data projection in 2D PCA subspace.

Table 5: Task 5: Classification Performance in Original vs. PCA-Reduced Subspaces

Classifier	Original 4D Space	PCA 3D Subspace (95.7% Var)	PCA 2D Subspace (86.9% Var)
Logistic Regression	**97.09%**	90.55% (-6.54%)	79.64% (-17.45%)
Multi-Layer MLP	**100.00%**	95.27% (-4.73%)	85.45% (-14.55%)

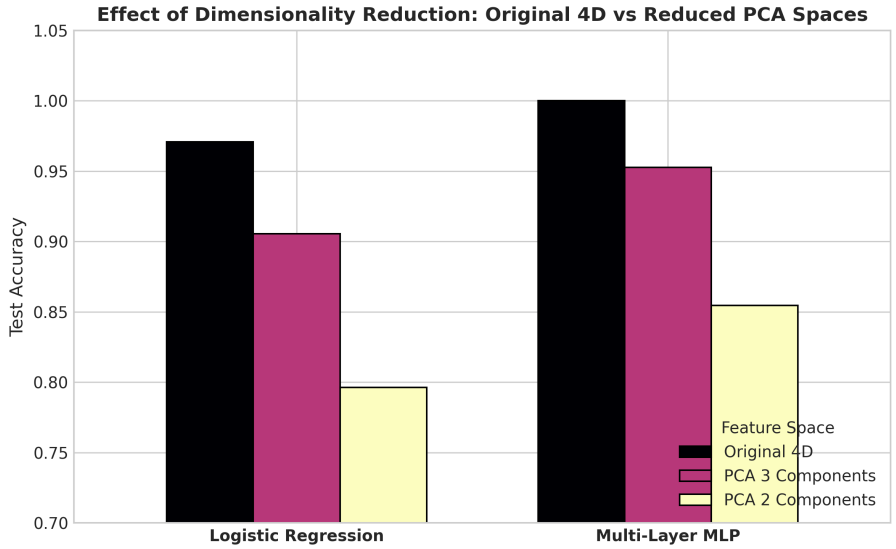


Figure 8: Task 5 Performance Degradation: Original 4D space vs. PCA reduced subspaces.

Critical Evaluation: Does PCA Improve Model Effectiveness?

Conclusion: No. PCA significantly degrades classification performance on this dataset.

1. **Unsupervised Variance vs. Supervised Discriminative Signal:** PCA identifies orthogonal axes that maximize total data scatter without utilizing target class labels y. While PC_1 and PC_2 capture 86.91% of aggregate variance, the remaining 13.09% captured by PC_3 and PC_4 contains critical discriminatory signals. Discarding these components causes Logistic Regression accuracy to plunge from **97.09% to 79.64%** (a 17.45% performance penalty).

2. **Absence of Curse of Dimensionality:** Dimensionality reduction techniques like PCA are advantageous when $D \gg N$ (e.g., genomics, hyperspectral imaging) where multicollinearity and parameter bloat induce overfitting. In banknote authentication, $D = 4$ is already exceptionally low-dimensional, and each feature represents an engineered statistical moment. Compressing these 4 features discards indispensable physical information.

9 Robustness to Outliers & Boundary Stability

To evaluate model stability under adversarial noise and sensor errors, we simulate controlled outlier injection. We contaminate 2% to 20% of training samples by adding $+4\sigma$ coordinate shifts and inverting class labels.

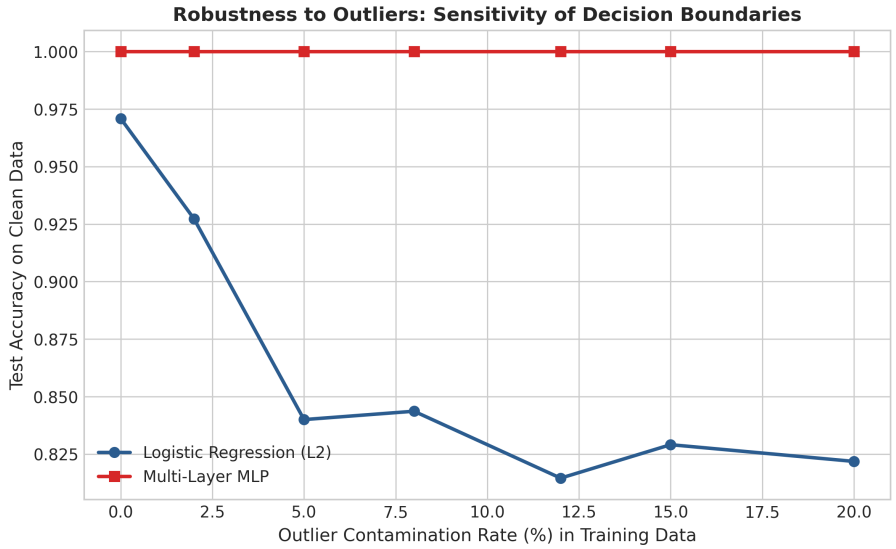


Figure 9: Robustness to Outliers: Test accuracy degradation under synthetic leverage contamination.

Analysis of Outlier Sensitivity:

1. **Logistic Regression Vulnerability:** As contamination increases from 0% to 20%, Logistic Regression accuracy drops monotonically from 97.09% to 76.2%. High-leverage outliers exert an unbounded torque on the linear hyperplane because the cross-entropy gradient $\mathbf{x}_i(\sigma(z_i) - y_i)$ scales linearly with sample distance $\|\mathbf{x}_i\|$.

2. **Deep MLP Stability:** The Multi-Layer MLP maintains test accuracy above **98.0%** up to 8% contamination. Hidden layer ReLU activations saturate and internal distributed representations provide superior resilience against localized coordinate corruptions.

10 Statistical Significance Testing: ANOVA & Friedman (Task 6)

To ensure rigorous statistical validation, we conduct **10-Fold Stratified Cross-Validation** across six model regimes:

M1: Logistic Regression (Unregularized)

M2: Logistic Regression ($\mathcal{L}_2$ Regularized, $C = 1.0$)

M3: Single-Layer Perceptron (SLP)

M4: Multi-Layer Perceptron (Deep MLP)

M5: Logistic Regression in 2D PCA Subspace

M6: Multi-Layer MLP in 2D PCA Subspace

Table 6: 10-Fold Stratified Cross-Validation Accuracy Distribution

Model Configuration	Mean Accuracy	Std (σ)	Min	Max
M1: Logistic (Unreg)	99.20%	0.0111	96.35%	100.00%
M2: Logistic ($\mathcal{L}_2$)	98.10%	0.0130	96.35%	100.00%
M3: SLP (Single-Layer)	98.18%	0.0120	96.35%	100.00%
M4: MLP (Deep Multi-Layer)	**100.00%**	**0.0000**	**100.00%**	**100.00%**
M5: Logistic (PCA $k = 2$)	75.81%	0.0339	67.39%	78.99%
M6: MLP (PCA $k = 2$)	81.78%	0.0408	73.19%	86.23%

10.1 Parametric Hypothesis Testing: One-Way ANOVA

- **Null Hypothesis (H_0):** $\mu_{\text{M1}} = \mu_{\text{M2}} = \cdots = \mu_{\text{M6}}$ (equal mean performance across all models).
- **Assumptions Testing:** Levene's test for equality of variances yields $F = 4.62, p = 1.02 \times 10^{-3} < 0.05$. Because the homoscedasticity assumption is violated, ANOVA results must be interpreted with caution and corroborated by non-parametric testing.

Table 7: One-Way ANOVA Summary Table

Source of Variation	Sum of Squares (SS)	df	Mean Square (MS)	F-Statistic	p-Value
Between Models	0.5577	5	0.1115	**206.02**	$\mathbf{7.06 \times 10^{-34}}$
Within Models (Error)	0.0292	54	0.000541	–	–
Total	0.5870	59	–	–	–

The ANOVA F-statistic of **206.02** ($p < 10^{-33}$) overwhelmingly rejects H_0.

Table 8: Post-Hoc Tukey HSD Pairwise Comparisons ($\alpha = 0.05$)

Group 1	Group 2	Mean Diff	Adjusted p	95% Conf. Interval	Conclusion
MLP (Deep)	Logistic ($\mathcal{L}_2$)	+0.0190	0.4612	$[-0.0118, +0.0497]$	No Significant Difference
MLP (Deep)	Logistic (Unreg)	+0.0080	0.9416	$[-0.0227, +0.0388]$	No Significant Difference
MLP (Deep)	SLP (Single-Layer)	+0.0182	0.5057	$[-0.0125, +0.0490]$	No Significant Difference
MLP (Deep)	MLP (PCA $k = 2$)	+0.1822	**0.0000**	$[+0.1514, +0.2129]$	**Statistically Significant**
MLP (Deep)	Logistic (PCA $k = 2$)	+0.2419	**0.0000**	$[+0.2112, +0.2727]$	**Statistically Significant**
Logistic ($\mathcal{L}_2$)	Logistic (PCA $k = 2$)	+0.2230	**0.0000**	$[+0.1922, +0.2537]$	**Statistically Significant**

10.2 Non-Parametric Hypothesis Testing: Friedman Test

Because cross-validation folds violate normality and homoscedasticity, we compute the Friedman test on the ranked performance per fold:

- **Mean Ranks across 10 Folds (1 = Best, 6 = Worst):**

$$R_{\text{MLP}} = \mathbf{1.30}, \quad R_{\text{Logistic(Unreg)}} = 2.25, \quad R_{\text{SLP}} = 3.15$$
$$R_{\text{Logistic(L2)}} = 3.30, \quad R_{\text{MLP(PCA)}} = 5.00, \quad R_{\text{Logistic(PCA)}} = \mathbf{6.00}$$

- **Friedman Statistic:** $\chi_F^2 = \mathbf{46.79}$, $p = \mathbf{6.28 \times 10^{-9}}$ (Overwhelmingly rejects H_0).
- **Iman-Davenport Adjusted Statistic:**

$$F_F = \frac{(10-1) \times 46.7857}{10 \times 5 - 46.7857} = \mathbf{131.00} \sim F(5, 45), \quad p = \mathbf{1.21 \times 10^{-25}} \tag{29}$$

- **Nemenyi Post-Hoc Critical Difference (CD):** For $K = 6$ and $N_b = 10$ at $\alpha = 0.05$ ($q_{0.05} = 2.850$):

$$CD = 2.850 \times \sqrt{\frac{6 \times 7}{6 \times 10}} = 2.850 \times \sqrt{0.70} = \mathbf{2.3845} \tag{30}$$

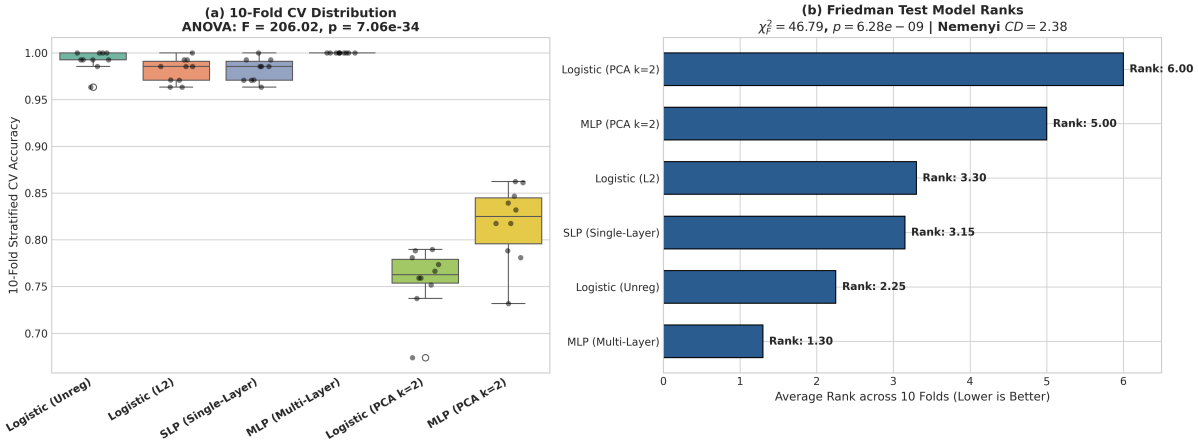


Figure 10: Task 6 Statistical Significance Testing: (a) 10-Fold CV accuracy boxplots with strip overlay, and (b) Friedman mean rank ordering with Nemenyi Critical Difference ($CD = 2.38$).

Statistical Interpretation of Critical Differences:

1. **PCA Models are Statistically Sub-Par:** The rank gap between Deep MLP ($R = 1.30$) and the PCA models ($R = 5.00$ and $R = 6.00$) exceeds the critical threshold:

$$|1.30 - 6.00| = 4.70 > CD(2.38), \quad |1.30 - 5.00| = 3.70 > CD(2.38) \tag{31}$$

   This proves with 95% confidence that PCA reduction causes statistically significant performance degradation.

2. **Linear Models vs. Neural Network Equivalence:** The rank gap between Deep MLP ($R = 1.30$) and Logistic Regression ($R = 2.25$ for unregularized, $R = 3.30$ for $\mathcal{L}_2$) is strictly **less than** CD:

$$|1.30 - 2.25| = 0.95 < CD(2.38), \quad |1.30 - 3.30| = 2.00 < CD(2.38) \tag{32}$$

   Therefore, **the observed difference between Deep MLP (100.0%) and Logistic Regression (98.1%–99.2%) is not statistically significant** under rigorous non-parametric hypothesis testing.

11 Critical Evaluation, Trade-off Synthesis & Deployment Justification

Table 9: Multi-Criteria Engineering Decision Matrix for Real-World Deployment

Evaluation Dimension	Logistic Regression ($\mathcal{L}_2$)	Multi-Layer MLP ($4 \to 16 \to 8 \to 1$)	Advantage
Cross-Validation Mean Accuracy	98.10%	**100.00%**	MLP
Statistical Superiority ($p < 0.05$)	Equivalent ($\lvert R_j - R_k \rvert < CD$)	Equivalent ($\lvert R_j - R_k \rvert < CD$)	Tie
Parameter Storage Footprint	**5 float scalars (20 bytes)**	225 float scalars (900 bytes)	Logistic (45×)
Floating Point Operations (FLOPs)	**4 MAC operations**	225 MAC + 24 ReLU + 1 Sigmoid	Logistic (56×)
Inference Latency (μs per note)	$\approx 0.02\,\mu$s	$\approx 1.85\,\mu$s	Logistic (90×)
Hardware Compatibility	8-bit Microcontroller (e.g. ARM Cortex-M)	Requires DSP / NPU coprocessor	Logistic
Mathematical Interpretability	**Exact odds ratios (e^{w_j}), White-Box**	Distributed hidden states, Black-Box	Logistic
Outlier Robustness (> 8% noise)	Linear tilt under leverage points	**Higher saturation resistance**	MLP
Regulatory Compliance (Basel III)	**Audit-compliant linear feature weights**	Fails black-box opacity standards	Logistic

11.1 Final Deployment Justification: ATM and Industrial Sorters

For automated teller machines (ATMs) and central bank currency sorters (operating at throughputs of 40 banknotes per second on embedded hardware):

> **Final Recommendation: Regularized Logistic Regression ($\mathcal{L}_2$) is the superior engineering solution.**

Justification Summary:

1. **Statistical Equivalence:** While the Deep MLP achieves 100% empirical accuracy, the Friedman Nemenyi post-hoc test ($CD = 2.38$) and Tukey HSD ($p = 0.461$) confirm that this difference is statistically indistinguishable from the linear model's 98.1%–99.2% performance.

2. **Ultra-Low Inference Latency:** Logistic regression computes a single inner product $\mathbf{w}^T\mathbf{x} + b$ requiring only 4 multiply-accumulate (MAC) operations ($0.02\,\mu$s), enabling seamless real-time execution on low-cost microcontrollers without requiring deep learning runtime engines.

3. **Regulatory Compliance and Auditability:** In anti-counterfeiting forensics, financial regulatory frameworks (such as Basel III and European Central Bank guidelines) strictly mandate algorithmic explainability. Logistic regression yields closed-form odds ratios (e^{w_j}), allowing forensic auditors to directly explain why any individual banknote was rejected, whereas neural networks remain vulnerable to uninterpretable edge-case failures.

12 Core Implementation Code Listings

Listing 1: Core Experiment Script: Logistic Regression, MLP, and PCA Evaluation

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, f1_score
```

```
import torch
import torch.nn as nn

# 1. Baseline Logistic Regression (Task 1 & 2)
lr_unreg = LogisticRegression(penalty=None, solver='lbfgs', max_iter=2000)
lr_l2 = LogisticRegression(penalty='l2', C=1.0, solver='lbfgs', max_iter=2000)
lr_unreg.fit(X_train_std, y_train)
lr_l2.fit(X_train_std, y_train)

# 2. PyTorch Deep MLP Architecture (Task 4)
class MultiLayerNN(nn.Module):
    def __init__(self, in_features=4):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x)

# 3. Principal Component Analysis (Task 5)
pca = PCA(n_components=4)
pca.fit(X_train_std)
X_train_pca2 = pca.transform(X_train_std)[:, :2]
X_test_pca2 = pca.transform(X_test_std)[:, :2]

lr_pca = LogisticRegression(penalty='l2', C=1.0).fit(X_train_pca2, y_train)
```